Introduction to Neural Network Interpretability

Part I: From Random Machines to Trained Machines

Contact Information
For permission requests or inquiries, please contact: ruchirtewari@gmail.com

Introduction to Neural Network Interpretability

Part I: From Random Machines to Trained Machines

Chapters 1–4

Ruchir Tewari

August 31, 2026

How to Read This book

This is the first half of a book that ends at the frontier of neural network interpretability. The destination - understanding and controlling what happens inside trained models (Part II, Chapters 5–8) - requires foundations that are usually scattered across information theory, differential geometry, and optimization. Part I collects them into a single arc:

1. **Chapter 1 - Statistical Foundations.** Where does structure in generated text come from? (Answer: not from randomness but from conditioning - the line from Markov's letter statistics through Shannon's entropy to the modern language-modeling objective.)
2. **Chapter 2 - Geometric Foundations.** How far apart are two probability distributions, honestly? (Answer: Fisher information turns distinguishability into curvature, and the Fisher–Rao metric turns families of distributions into a landscape with true distances.)
3. **Chapter 3 - Neural Networks as Distribution Machines.** What, mathematically, does a trained network do? (Answer: it shapes probability distributions - sequentially by conditioning in transformers, globally by warping in flow and diffusion models, and in parallel by splitting the job across heads and experts.)
4. **Chapter 4 - Training Dynamics.** How does the machine get tuned? (Answer: by descending a loss landscape whose true geometry is the Fisher metric - explicitly in natural-gradient methods, implicitly in the flat-minima bias of plain stochastic gradient descent.)

The two threads - statistical (Chapters 1, 3) and geometric (Chapters 2, 4) - braid together deliberately. Chapter 1 explains *what* a generative model must produce; Chapter 2 explains how to *measure* progress toward it; Chapter 3 shows the machines that produce it; Chapter 4 shows how measurement geometry governs the tuning of those machines. Part II then opens the trained machine and asks what formed inside.

Some conventions, shared with Part II:

- Citations appear inline as (Author, Year); full references, with the filenames of archived PDFs in `new_book/references/`, are collected in the References chapter at the end.
- Short **Neuroscience parallel** paragraphs appear inline where an idea has a direct counterpart in the study of biological brains - stated concretely: what the object is in the brain, and what the corresponding object is in the artificial network.
- Chapters close with three collected sections: **Takeaways**, **Research Directions**, and **Programs** - hands-on projects that let you reproduce the chapter's main claims yourself. All programs run on a laptop with Python 3, `numpy`, `matplotlib`, and (for Chapters 3–4) `torch`.
- No result is presented without saying what kind of evidence supports it. In Part I the mathematics is mostly settled; the places where claims are empirical rather than proved (notably in Chapter 4) are flagged explicitly.

Contents

1. Statistical Foundations

In *One Two Three… Infinity* (1947), George Gamow supposes a printing press that prints letters automatically one after another which given enough time would produce every line from Shakespeare. In his construction, the letter producing dials move deterministically as a car meter dial turns the next dial when completing a turn. To produce all possible character combinations of an 80 character line would require printing $27^{80} = 10^{114}$ combinations. If instead the machine were built to output randomly on each line a sequence of 10 words from Shakespeare's vocabulary of about 15000, there would be a smaller number of 15000^{10} or 10^{42} combinations. If the machine were to write a line a billion times a second for a billion years, it would produce only 10^{26} lines and would be far from writing every possible line with either character or word combinations.

Now suppose the machine were driven by a random number generator with a controlled probability distribution of the production of the words. The probability distribution could model the frequency of the words in the language, the frequency of two or three word combinations, and be able to learn patterns in Shakespeare according to a probability distribution. The output by the machine would significantly improve on the odds of producing text that looked like Shakespeare, yet produce gibberish text most of the time. The word *model* is as a synonym for a probability distribution. A modern language model is an answer to the question the text producing machine raises: what has to be added to randomness before it starts producing language?

1.1 The statistical machine

The printing press samples from a probability distribution that carries *zero information about language*. Improving the press means changing the distribution - concentrating its mass on the vanishingly small subspace of strings that look like language.

1.1.1 Markov reads Pushkin

The Russian mathematician Andrei Markov in 1913 took the first 20,000 letters of Pushkin's verse novel and counted, by hand, how often a vowel was followed by a vowel, a vowel by a consonant, a consonant by a vowel, and a consonant by a consonant (Markov, 1913). The counts were dramatically non-uniform: a vowel followed a consonant far more often than it followed another vowel. Letters in real text are not independent draws - each letter carries statistical information about its neighbor.

Markov's motivation was a technical dispute about whether the law of large numbers applies to dependent events. *Eugene Onegin* was a convenient sequence of dependent events. The byproduct was the founding observation of statistical language modeling. Language has *local statistical structure*. That structure is *measurable by counting*. And a machine equipped with those counts is no longer searching a flat space - it searches a space already shaped like the target. Markov showed letters are not independent in their statistics and one does not need the entire history to make a prediction about the next letter.

1.2 The Markov Property

1.2.1 Memorylessness, made precise

A stochastic process has the **Markov property** when its next state depends only on its current state, not on the path that led there:

$$P(X_{t+1} \mid X_t, X_{t-1}, \ldots, X_1) \;=\; P(X_{t+1} \mid X_t).$$

The word "memoryless" is standard but slightly misleading: the process does have memory - one state's worth. The design question that drives the rest of this chapter is: *what should the state be?* Markov's Pushkin analysis used the smallest interesting state, the class (vowel/consonant) of the current letter. Enriching the state is the single move that separates every generation of language model from the previous one.

An order-k (or "n-gram") model takes the state to be the last k symbols: the next character is drawn from the empirical distribution of what followed those k characters in a training corpus. An order-0 model draws each symbol independently from the corpus letter frequencies.

1.2.2 Climbing the ladder

Shannon (1948) made the ladder vivid by generating text from successively richer approximations to English, and the samples remain the fastest way to feel what each increment of context buys:

- **Zero-order** (the uniform distribution; Shannon's naming starts here): *XFOML RXKHRJFFJUJ ZLPWCFWKCYJ…* - noise.
- **First-order** (letter frequencies only): *OCRO HLI RGWR NMIELWIS EU LL…* - the texture changes; vowels appear at roughly English rates, but nothing is pronounceable.
- **Second-order** (letter pairs, Markov's statistics): *ON IE ANTSOUTINYS ARE T INCTORE ST…* - pronounceable fragments emerge; word-like units appear.
- **Third-order** (letter triples): *IN NO IST LAT WHEY CRATICT FROURE…* - most units could be English words; some are.
- **First-order word model**: *REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME…* - real words, no grammar.
- **Second-order word model**: *THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHARACTER OF THIS…* - locally grammatical stretches several words long.

Each rung conditions on more context, and each rung's output is recognizably closer to language. An interactive companion to this ladder is at `html/sentence-generators.html`: uniform and Markov sentence generators side by side, with dials for the conditioning, so you can watch the samples change as structure is added.

1.2.3 The state-explosion wall

The ladder cannot be climbed indefinitely. An order-k character model over a 27-symbol alphabet has 27^k possible states. An order-k *word* model over a 50,000-word vocabulary has $50{,}000^k$. At $k = 5$ words, that is 3×10^{23} contexts. Almost none of them ever occur, even in an internet-scale corpus. A context that never occurs contributes no counts, and a next-word distribution cannot be estimated from zero counts. Conditioning on long contexts by counting is therefore statistically impossible: every long context is new. Call this the **state-explosion wall**. It is not unique to language modeling. Other fields hit the state-explosion wall and the ways they got past it serve as directional hints, even where the full mechanism is still not understood.

- **Statistical mechanics.** A gram of gas contains $\sim 10^{22}$ molecules. Its space of microstates dwarfs $50{,}000^5$ beyond comparison. Boltzmann and Gibbs got past the explosion by refusing to track microstates at all. They tracked a handful of macroscopic variables (temperature, pressure, entropy) that summarize everything about the microstate that matters for prediction.
- **Data compression.** A compressed file is short for one reason: the source is not uniform. A 27-symbol alphabet costs 4.75 bits per character if coded naively; because English's statistics are lopsided, about 1 bit per character suffices. Shannon's source-coding theorem says compressibility comes from probability being extremely nonuniform over possibility space and this is exploited in compression algorithms such as Huffman coding which gives short bit strings to common symbols and longer bit strings to rare symbols, and converts probability into code length.
- **Biology.** An oak tree contains on the order of 10^{12} cells, arranged in structures no blueprint could enumerate. The acorn stores none of that arrangement. It stores a compact generative program (a genome of a few gigabits) plus a developmental process that rebuilds the structure on demand. The escape move: store a *generator* of the structure, not the structure itself.

The repeated solution: when states explode, find the compact object (sufficient statistics, the typical set, a generative program) that produces the needed predictions without a table of states. Language modeling after the n-gram era reached for the same solution. Its compact object is a *learned state*: a vector-valued summary of the context, produced by a trained network, that generalizes across contexts instead of counting them.

1.3 Measures over outcomes and distributions

1.3.1 Surprise and its expectation, Entropy

Shannon's 1948 paper built on the ladder samples to define the quantity that measures what the ladder is climbing toward. Define the **information content** or **surprise** of an outcome x with probability $p(x)$ as $-\log_2 p(x)$, measured in **bits**: an outcome with probability $\frac{1}{2}$ carries 1 bit; probability $\frac{1}{1024}$ carries 10 bits. Surprise is a measure for a single outcome. Consider two scenarios of a weather forecast. Forecast said 90% sun, and it was sunny : the surprise is $-\log(0.9) = 0.15$ bits - it is expected, barely register it. Forecast said 90% sun, but rain happened: this is $-\log(0.1) = 3.32$ bits of surprise, one is shocked.

Entropy is the probability weighted average of the bits required for each outcome in a distribution:

$$H(p) \;=\; -\sum_{x} p(x)\, \log_2 p(x). \tag{1.1}$$

It is a measure over a distribution, not a single event. A distribution has a number of outcomes with different probabilities. Entropy takes the number of bits required per outcome, and then weights that number of bits by the probability itself and adds these to get the everage bit needs to encode any outcome in the distribution. It is the expectation of the surprise or E[surprise].

For a given number of outcomes, entropy is maximized by the uniform distribution and shrinks as probability mass concentrates. For a distribution with n outcomes each of probability 1/n, entropy is $n*(1/n)*log_2(n) = log_2(n)$. This increases with n, and makes entropy a measure of spread - for sets of outcomes of equal probability each, the larger the set of outcomes, the higher the entropy. It is a rigorous measure of "how unpredictable the next symbol is". When measured in base 2 logarithm, it has the unit of bits, and using natural logarithms, it has the unit of nats.

Each next rung of the ladder conditions on more context and therefore faces less uncertainty per character.

1.3.2 Cross-entropy and Perplexity

A code is a rule that converts each outcome into a sequence of bits. Now a code can be efficient for a distribution if it produces bit lengths that match the information content of the outcome derived from their probability. In general this is not true - let's take an example. Say we have 4 bit codes 0000, 0001 to 1111 that encode 16 outcomes that are not equally probable and that are in fact highly skewed. Say 4 outcomes have 22% probability each for a total of 88% and the remaining 12 outcomes have a 1% probability each. The entropy calculation shows that this requires 2.37 bits on average, but we are paying 4 bits per outcome. This motivates a measure called cross-entropy.

Cross-entropy is a measure of the difference between two probability distributions. A model that produces probabilities according to a distribution q trying to predict text that is actually distributed as p pays the following number of bits on average:

$$H(p, q) \;=\; -\sum_{x} p(x) \log_2 q(x) \tag{1.2}$$

Note this is not symmetric.

If the two distributions p and q are equal, cross-entropy reduces to the entropy of q (and not to zero).

Perplexity, is defined as the exponentiation of cross-entropy - $2^{H(p,q)}$

The difference between the cross-entropy and the entropy of the target distribution p, is the divergence of p with respect to q, and this goes to zero as q approaches p. Minimizing cross-entropy on training text is the objective by which GPT-class models are trained and the objective is reached when the divergence is zero or minimum - training consists of descending that scale by reshaping q toward p. Subtracting equation 1.1 from equation 1.2 we get another measure of the difference between two distributions:

$$H(p, q) - H(p) = D(p \,\|\, q) \tag{1.3}$$

This divergence is not symmetric either. There are other measures of distances between probability distributions that are symmetric such as JS divergence that are discussed later.

The companion page (`html/sentence-generators.html`) scores each of its generators by cross-entropy against Shakespeare, so the entropy ladder and the perplexity numbers of this section can be explored by hand.

Neural networks can be thought to repeatedly transform broad probability spaces into highly concentrated internal distributions. A key is internal representations, which this book is building towards understanding.

Neuroscience parallel. Sensory neurons face the entropy problem too. Barlow (1961) proposed the **efficient coding hypothesis**: early sensory neurons recode their inputs to remove statistical redundancy. Spikes are metabolically expensive and channel capacity is limited, so a neuron should spend its signaling on the surprising part of the input, not the predictable part. Laughlin (1981) confirmed a sharp version of this prediction in the blowfly eye. He measured the contrast-response curve of a specific visual interneuron and compared it to the statistics of natural scenes. The neuron's response curve follows the cumulative distribution of contrasts found in nature, allocating its limited output range where the input probability mass actually is.

Programs for Chapter 1

`uniformrandom_vs_markov_text.py` Download a public-domain text (Project Gutenberg Shakespeare works). Generate 500 characters each from: uniform random; order-0 (letter frequencies); order-1 through order-4 character Markov models built by counting; order-1 and order-2 word models. Print the samples as a ladder, and for each, report per-character cross-entropy against held-out text. You will reproduce Shannon's 1948 table on your own corpus and watch language condense out of noise, one order at a time. *Type: script, prints ladder + entropy table. Deps: numpy only.*

`entropy_vs_context_length.py` Using the same corpus, estimate conditional entropy $H(X_t \mid X_{t-k}, \ldots, X_{t-1})$ for $k = 0 \ldots 8$ characters by counting (with add-one smoothing), and plot the falling curve. Mark Shannon's reference points (4.75, ∼4.1, ∼3.3, ∼1.0 bits). Then show the estimator break: plot the number of distinct contexts observed vs. 27^k - the state-explosion wall of §1.2 appearing as data sparsity in your own counts. *Type: script + matplotlib. Deps: numpy.*

2. Geometric Foundations

We want to adjust the probability distributions that are generated to get them closer to a target. Entropy tells you how uncertain a machine's next symbol is, but does not give a way to control it. To do this we add *dials* to the machine, that change how the output distribution responds when you nudge them. Training is dial-nudging a few million or billion knobs at a time. This chapter builds the mathematics of dial-sensitivity, and it is genuinely a second notion of "information," due to Fisher-Rao rather than Markov-Shannon.

A picture to hold throughout, extending Gamow's typing machine: imagine a whole *colony* of letter machines, each with slightly different dial settings - one biased 40% toward vowels, another 41%, another 60%. Two machines whose outputs are nearly indistinguishable should count as "close," whatever their dial labels say; two machines with obviously different output should count as "far," even if their dials differ by a hair. The mathematics of this chapter is the construction of that honest ruler.

Structure of this chapter. Section 2.1 defines **Fisher information** through the most concrete picture available: nudge a parameter by δ and ask how distinguishable the new output distribution is from the old one; the answer is a bowl-shaped curve whose curvature at the bottom *is* the Fisher information. Section 2.2 assembles the bowls (one at every parameter setting) into the **Fisher–Rao metric**, a Riemannian geometry on the space of distributions, with geodesics as true shortest paths and a uniqueness theorem explaining why this metric and no other. Section 2.3 sets Shannon and Fisher side by side and shows they measure genuinely different things, uncertainty of *outcomes* versus estimability of *parameters*, which can point in opposite directions for the same distribution. Chapter 4 will put this geometry to work in training.

2.1 Fisher Information as Curvature

2.1.1 The ruler first: Kullback–Leibler divergence, defined

Let's look at divergence again. For two distributions p and q over the same outcomes, define the KL divergence as

$$D_{\mathrm{KL}}(p \,\|\, q) \;=\; \sum_{x} p(x) \log \frac{p(x)}{q(x)} \tag{2.1}$$

It follows from 1.3 and is the average, *taken under* p, of the log-ratio between the two probability assignments. The double-bar notation "$\|$" is a separator, not an operation: it marks which distribution plays which role. The distribution on the *left* of the bar is the reference - the one assumed to actually generate the data, the one the expectation averages over; the distribution on the *right* is the one being judged against it. Read $D_{\mathrm{KL}}(p \,\|\, q)$ as "the divergence *of* q *from* p," or operationally: the expected number of extra bits paid per outcome when outcomes truly drawn from p are encoded using a code built for q.

The order matters because the two roles are not interchangeable: $D_{\mathrm{KL}}(p \,\|\, q) \neq D_{\mathrm{KL}}(q \,\|\, p)$ in general and the divergence is not symmetric. Additionally it violates the triangle inequality,

and is therefore *not a distance*. The two orderings ask different questions - "how surprising is q's code to a p-world?" versus the how surprising p's code is to a q world .

However for *infinitesimally* separated distributions the asymmetry vanishes, and a genuine symmetric ruler emerges from an asymmetric one.

2.1.2 From surprise to the score to Fisher information

We defined the surprise of an outcome x as $-\log p_\theta(x)$: how shocked the machine is to see x. Written with the sign flipped, $\log p_\theta(x)$ is the **log-likelihood** of x under the model.

Now ask how the shock responds to the dials. Take a parametrized family p_θ (the vowel-bias dial on a letter machine, the many weights in a transformer) and differentiate the log-likelihood with respect to θ. This gradient of the surprise with respect to the model weight is called the **score**, s(x):

$$s(x) \;=\; \nabla_\theta \log p_\theta(x),$$

It is a vector in the parameter space, and points in the direction that makes x more likely - the direction θ would move if x alone were the training set. Different outcomes x vote for different directions.

The log goes on *before* the gradient for two structural reasons. First, products become sums: a sequence factorizes as $p_\theta(x_{1:T}) = \prod_t p_\theta(x_t \mid x_{<t})$, and the log turns the product into a sum whose gradient is just the sum of per-token scores. Second, the log makes the gradient measure *relative* change: $\nabla_\theta \log p = (\nabla_\theta p)/p$, the raw sensitivity normalized by the current probability. What matters for information is not how much p moves absolutely but how much it moves relative to where it stands.

Averaged over the model's *own* outcomes, the votes cancel, or the expectation of the score is zero.

$$\mathbb{E}_{p_\theta}[s] \;=\; \sum_x p_\theta(x)\, \nabla_\theta \log p_\theta(x) \;=\; \nabla_\theta \sum_x p_\theta(x) \;=\; \nabla_\theta 1 \;=\; 0.$$

This is because a machine that fed the data it expects asks for no parameter change. Each score may be non-zero but the average score is zero as the model matches reality - if it did not, this would be the training gradient.

While mean is zero, we can still look at the variance - there is information in the spread. Define the covariance of the score as follows:

$$F(\theta) \;=\; \mathbb{E}_{x\sim p_\theta}\Big[s(x)\, s(x)^\top\Big],$$

This covariance of the score is called the **Fisher information matrix**. Read the covariance as disagreement among outcomes. Large F: different observations push θ hard in different directions - each outcome reveals something distinct about the dials, and the output is informative about the machine that produced it. Small F: every observation pushes weakly or in the same direction - moving θ barely changes which outcomes are likely, and the dial hides. Fisher information is how much a single observation can tell you about the parameters, or equivalently how sensitive the model's predictions are to parameter changes, averaged over what the model itself expects to see.

The divergence arrives at the same matrix. Nudge the parameter from θ to $\theta+\delta$ and measure distinguishability with the divergence of the previous subsection, $g(\delta) = D_{\mathrm{KL}}(p_\theta \,\|\, p_{\theta+\delta})$, Taylor-expanded around $\delta = 0$: $g(\delta) \approx g(0) + g'(0)\,\delta + \frac{1}{2}\, g''(0)\,\delta^2$. The first two terms vanish. The zeroth-order term is $g(0) = D_{\mathrm{KL}}(p_\theta \| p_\theta) = 0$; the first-order term vanishes because $\delta = 0$ is a

minimum, and the slope there is zero - precisely block 2's anchor, the expected score being zero, wearing its geometric hat. The leading survivor is the quadratic term, with Taylor's standard $\frac{1}{2}g''(0)$ coefficient, and $g''(0)$ is the covariance just built. This is the central formula of the chapter:

$$D_{\mathrm{KL}}(p_\theta \,\|\, p_{\theta+\delta}) \;\approx\; \tfrac{1}{2}\,\delta^\top F(\theta)\,\delta.$$

One object, two constructions: from the inside, F is the covariance of per-observation parameter votes; from the outside, it is the curvature of distinguishability under a nudge. The two constructions agree because both are second derivatives of the same underlying surprise.

Notice also what the expansion delivered: $\delta^\top F\delta$ is symmetric in $\delta \to -\delta$, and expanding the KL divergence in the *other* order, $D_{\mathrm{KL}}(p_{\theta+\delta}\|p_\theta)$, gives the *same* quadratic form with the same F. The asymmetry of KL is a third-order effect. Locally - and only locally - distinguishability is symmetric, which is exactly what a metric requires and why Section 2.2 can build one.

2.1.3 Which bowl? Getting the picture right

The quadratic $\frac{1}{2}\delta^\top F\delta$ is a bowl, and the bowl image recurs so let's be precise now about which bowl this is. On the horizontal axes, lies the *nudge* δ of the paramters at their current value - not the parameters themselves a their target distribution (p). On the vertical axis, the divergence between the original machine and the nudged one. The bottom of this bowl sits at $\delta = 0$ with height exactly zero, *always, by construction, at every parameter setting* θ - at a well-trained optimum, at a terrible initialization, anywhere. There is one such bowl at every point of parameter space, and its shape (not its location) holds information: steep narrow walls mean a hair-trigger dial whose tiniest turn produces detectably different output (high Fisher information); wide shallow walls mean a loose dial you can turn without anyone noticing (low Fisher information). It is a *curvature*, not a slope - the slope at the bottom is zero by construction.

This is *not* the loss bowl of stochastic gradient descent. The training-loss surface is a function of the parameters themselves, with the data held fixed; its minimum sits wherever training happens to converge, its depth and slope carry meaning, and there is one landscape for the whole problem. The KL bowl is a function of a *hypothetical nudge*, re-centered at whatever θ you currently occupy, and its minimum is always trivially zero. The two are related - for log-loss, the expected Hessian of the loss under the model's own distribution equals F, which is why the pictures rhyme and why Chapter 4 can use F as a stand-in for loss curvature - but confusing them costs you the key property: the loss bowl tells you *how wrong* you are; the KL bowl tells you *how sensitive* you are, and sensitivity is defined even where wrongness isn't.

Is a sharp bowl good or bad? The question has no fixed answer, and that is worth internalizing: Fisher information is *descriptive*, a sensitivity, and its valence depends on who is asking. For an *estimator* trying to identify the dial from output, sharp is good - the parameter is pinned precisely (next two subsections). For *robustness*, sharp is bad - a hair-trigger direction is one where tiny parameter noise, quantization error, or a fine-tuning step visibly changes behavior. For an *optimizer*, sharp means "step carefully": Chapter 4's natural gradient divides by F precisely to take small steps in sharp directions and bold steps in flat ones. Same number, three readings.

2.1.4 Whose probabilities? What the network "knows"

A network is a pile of matrix multiplications; so in what sense does it *have* a p_θ for KL to compare - how does it know these numbers are probabilities ? The answer is: by construction, at the output. A language model's final layer produces a vector of real numbers (logits), and the **softmax** function exponentiates and normalizes them into nonnegative numbers summing to one - a probability distribution over the vocabulary, whether or not anything "believes" it. The training objective then makes the label stick: cross-entropy loss rewards exactly the calibration of those numbers against observed next tokens, so a trained model's outputs are not merely normalized but *meaningfully* probabilistic - they are the quantities the objective sharpened. For a whole sequence the model assigns $p_\theta(x_{1:T}) = \prod_t p_\theta(x_t \mid x_{<t})$, each factor a softmax output. The network knows nothing; the architecture guarantees normalization, and the objective supplies the semantics. $D_{\mathrm{KL}}(p_\theta \| p_{\theta+\delta})$ is then a statement about the two machines' output distributions - the observable channel - not about any hidden interpretation.

This resolves a question the formula quietly raises: F is defined by an expectation over $x \sim p_\theta$ - outcomes drawn from *the model itself*, not from training data. Fisher information is a property of the machine at its current setting, measurable in principle by letting the machine run and watching how its own outputs vote.

2.1.5 The score and the network Jacobian

For the letter machine, $\nabla_\theta \log p_\theta(x)$ is one derivative. For a transformer it decomposes, and the decomposition is where the geometry meets the architecture. Write the model as parameters $\theta \mapsto$ logits $z(\theta) \mapsto$ probabilities $p = \mathrm{softmax}(z)$. By the chain rule, the score is

$$s(x) \;=\; \nabla_\theta \log p_\theta(x) \;=\; J_z^\top \, \nabla_z \log p(x),$$

where $J_z = \partial z / \partial \theta$ is the **Jacobian** of the logits with respect to the parameters - the object backpropagation computes. Substituting into the covariance form,

$$F(\theta) \;=\; \mathbb{E}\Big[J_z^\top \, G(z) \, J_z \Big], \qquad G(z) = \mathrm{diag}(p) - p\,p^\top,$$

the Fisher matrix of the softmax itself, conjugated by the network Jacobian. Read it as a *pullback*: the output simplex carries a fixed, known information geometry (G), and the network's Jacobian pulls that geometry back onto the weight space - each parameter direction inherits exactly as much curvature as it can move the output distribution. Directions the Jacobian kills (parameter changes the output not at all) get zero Fisher information; directions the Jacobian amplifies get more. This form - known output geometry composed with a learned Jacobian - is also the computational workhorse: it is the structure that the approximations below exploit.

2.1.6 Two dials you can compute by hand

The biased coin. For a Bernoulli distribution with heads probability p,

$$F(p) \;=\; \frac{1}{p(1-p)}.$$

At $p = 0.5$ the Fisher information is 4; as $p \to 0$ or 1 it diverges. A fair coin is the *loosest* dial (nudging 0.50 to 0.51 changes the output stream imperceptibly), while near-deterministic

coins are hair-triggers: at $p = 0.99$, the same 0.01 nudge to 1.00 eliminates tails entirely, an unmistakable change. Distinguishability per unit of dial is not constant across the dial's range, and that non-constancy is the whole reason a geometry is needed.

The Gaussian mean. For a Gaussian with known standard deviation σ, the information about the mean is $F(\mu) = 1/\sigma^2$. A narrow distribution pins its mean sharply - every sample is a precise vote; a wide one leaves the mean blurry. Hold this example: it returns in Section 2.3 as the cleanest demonstration that Fisher and Shannon disagree.

2.1.7 Why estimators care: the Cramér–Rao bound

Fisher information earns its name from estimation theory. The **Cramér–Rao bound** states that any unbiased estimator $\hat{\theta}$ built from n samples has variance

$$\mathrm{Var}(\hat{\theta}) \;\geq\; \frac{1}{n\,F(\theta)}.$$

The bowl's curvature sets the best achievable precision for pinning down the dial from output alone. This is the lie-detector reading of the same mathematics. You watch output from one of two secretly different machines. Your ability to tell which machine you face is governed by F. High curvature: the output betrays its parameter. Low curvature: the parameter hides. The bound applies to any reader whatsoever - human analyst, statistical test, or trained classifier. Any attempt to decode a hidden quantity from a system's observable behavior operates under this ceiling, whatever the decoder.

2.1.8 Measuring it in a billion-dial machine

For the coin and the Gaussian, F came out in closed form. For a transformer there is no analytic formula - the expectation runs over the model's own output distribution through a billion-parameter Jacobian - and worse, the full matrix could not even be *stored*: at d parameters it has d^2 entries, which for $d \sim 10^9$ is 10^{18} numbers. Fisher information for networks is therefore always *estimated* and always *structured*. The standard moves, in increasing order of approximation:

- **Monte Carlo, exactly.** The covariance form is an expectation over $x \sim p_\theta$, so sample: run the model, draw outputs from its own distribution, backpropagate $\log p_\theta(x)$ to get one score vector per sample, and average outer products. This is unbiased for the true F at any sample budget - the estimate is noisy, never wrong on average.
- **Fisher–vector products.** Most uses need Fv for chosen directions v, not F itself - and Fv is computable by automatic differentiation without ever materializing the matrix, the same trick that makes Hessian-vector products cheap. Storage objection dissolved.
- **The empirical Fisher.** Replace samples from the model with training examples and their labels - averaging outer products of the loss gradients you were computing anyway. Nearly free, widely used, and subtly *different*: at a poorly fit point the empirical and true Fisher can disagree badly, a distinction that matters in practice and is often ignored.
- **Structured approximations.** Keep only the diagonal (one sensitivity per parameter), or per-layer Kronecker-factored blocks (K-FAC), trading fidelity for tractability.

What is the calculation *for*? Even where exact computation is infeasible, F earns its keep in four ways. Practically: Chapter 4's **natural gradient** and its K-FAC implementations divide the gradient by (approximate) F, taking cautious steps along hair-trigger directions - the bowl put to work. Practically again: *elastic-weight-consolidation*-style methods use diagonal F as a per-parameter importance score, penalizing movement along high-information directions to avoid catastrophically forgetting old tasks - and the same importance reading ranks which parameters matter for a behavior. Theoretically: the Cramér–Rao bound holds whether or not anyone computes F - it is a ceiling on every probe and decoder in Part II, a statement about what the observable channel *can* reveal. And diagnostically: the spectrum of F (even sampled crudely) characterizes the local geometry of training - how many stiff directions, how many sloppy ones - which Part II's Chapter 8 uses to watch training reorganize a network's representation before its behavior shows it.

2.2 The Fisher–Rao Metric and Manifold

2.2.1 From one bowl to a landscape

Section 2.1 built one bowl at one dial setting. Now build one at *every* setting and stitch them together: declare that the squared length of an infinitesimal parameter step $d\theta$ is

$$ds^2 \;=\; d\theta^\top F(\theta)\, d\theta.$$

This makes the parameter space of the family a **Riemannian manifold**, a curved space where the local ruler varies from point to point, with the Fisher matrix as its metric tensor. Distances measured by this ruler count *accumulated distinguishability*: a path is long if the distribution changes detectably along it, short if the machine sounds the same the whole way, regardless of how far the raw dial numbers traveled. This is precisely the honest ruler the machine colony needed (Nielsen, 2018, gives a modern self-contained construction).

The payoff properties, each of which fails for the naive Euclidean ruler on dial values:

- **Reparametrization invariance.** Relabel the dial (percent vowels, log-odds of vowels, anything invertible) and all Fisher–Rao distances are unchanged. The geometry belongs to the family of distributions, not to your coordinates on it. Euclidean distance on dial values changes wildly under the same relabeling.
- **Geodesics.** The manifold has shortest paths, called **geodesics**, which bend, in dial coordinates, to route through regions where the bowls are shallow (cheap to cross) and around regions where they are steep. For the Bernoulli family the geodesic distance has a closed form,
$$d_{FR}(p,q) \;=\; 2\arccos\!\big(\sqrt{pq}+\sqrt{(1-p)(1-q)}\big),$$
which stretches near the endpoints of the dial, just where $F = 1/p(1-p)$ blows up: moving a coin from 0.98 to 0.99 is a *long* journey in distinguishability, though a short one on the dial. The program `fisher_rao_distance_bernoulli.py` plots this warping directly.
- **Uniqueness.** Chentsov's theorem (1972) singles the Fisher–Rao metric out: up to scale, it is the *only* Riemannian metric on families of distributions invariant under natural probabilistic maps (Markov morphisms - coarsenings and relabelings of outcomes). There is one honest ruler, not a menu of candidates. The invariance class is named for the same

Andrei Markov as Chapter 1's property, and the link is real: Markov morphisms are built from the transition kernels his chains introduced.

2.2.2 Why a book about interpretability starts here

Two reasons. The first is training. Chapter 4's account of optimization is that training navigates this manifold: natural-gradient methods steer by the Fisher metric explicitly, and plain stochastic gradient descent turns out to respect it implicitly. None of that story can be stated without this chapter. The second is discipline. The recurring failure mode in analyzing any high-dimensional system is trusting coordinates (dial labels, neuron indices) instead of invariants. This chapter is the inoculation. Whenever this course measures an internal space of a trained network, the operating rule is the one built here: treat spaces of distributions and representations as geometric objects, and measure them with rulers that do not change when the labels do.

2.3 Contrasting Shannon and Fisher

2.3.1 Two questions, two informations

Both quantities are called "information," and the collision of names causes real confusion. They answer different questions about different objects:

- **Shannon entropy** is a property of *one distribution*: how uncertain is the outcome? It lives on the outputs.
- **Fisher information** is a property of a *family* of distributions at a point: how sharply does the distribution change as the parameter moves? It lives on the dials.

The Gaussian-mean example held from Section 2.1 shows they can point in opposite directions for the same object. Large σ: entropy is *high* (outcomes very unpredictable) while Fisher information about the mean, $1/\sigma^2$, is *low* (the mean is hard to estimate - the distribution barely changes shape when you slide it). Small σ: entropy low, Fisher information high. Unpredictable output and informative-about-parameters output are different properties; a distribution can spread wide over outcomes while pinning its parameters loosely, and vice versa.

	Markov/Shannon	**Fisher/Rao**
Core object	sequences, messages, one output distribution	a parametrized family of distributions
Key quantity	entropy, conditional probability, cross-entropy	Fisher matrix, Fisher–Rao distance
Question	how surprising is the outcome?	how distinguishable are nearby machines?
Lives on	outputs	parameters
Used for (this course)	the training *objective*; the n-gram ladder (Ch. 1, 3)	the training *geometry*; flat-minima analysis (Ch. 4, 8)
Zero means	outcome is certain	dial is undetectable from output

The one-line division of labor for everything that follows: *Shannon says what the model should say; Fisher says how the model should move to learn to say it.* Chapter 3 builds the machines whose outputs Shannon scores; Chapter 4 shows their tuning governed by Fisher.

Neuroscience parallel. Fisher information is a working tool of sensory neuroscience. There, the "dial" is a stimulus variable and the "machine" is a population of noisy neurons. Seung and Sompolinsky (1993) computed the Fisher information such a population carries about a stimulus, such as motion direction. The Cramér–Rao bound then gives the best decoding precision any downstream brain area can achieve. This turned a vague question (how accurately can the animal know the stimulus?) into a computable property of measured tuning curves. The same accounting applies unchanged to an artificial network. The stimulus variable is the parameter. The activation vector is the population response. Any classifier trained to read the activations is the decoder whose ceiling the bound sets. One formula prices both codes.

Takeaways

- Fisher information is the *curvature* of the Kullback–Leibler bowl around a parameter setting (distinguishability per squared unit of dial-turn) and, via Cramér–Rao, the ceiling on how precisely output can reveal its parameter.
- Stitching the bowls together yields the Fisher–Rao metric: reparametrization-invariant, geodesic-equipped, and unique by Chentsov's theorem - the one honest ruler on a family of distributions (Nielsen, 2018).
- Shannon and Fisher measure different things (outcome uncertainty vs. parameter estimability) and can disagree on the same distribution (the wide Gaussian). The course keeps them in separate columns: objective vs. geometry.

Research Directions

From §2.1: The probe ceiling in practice

A common analysis technique is the linear probe: a classifier trained to read some concept out of a network's internal activations. The Cramér–Rao bound prices any such readout, but nobody has systematically measured how close real probes come to the ceiling. On a small model, pick a controllable input concept (sentiment of a template, language of a word), estimate the Fisher information the activations at each layer carry about the concept parameter, and compare probe accuracy against the bound, layer by layer. The gap decomposes into "information absent" vs. "readout suboptimal" - a decomposition that claims about networks constantly conflate.

From §2.2: Fisher–Rao distances between checkpoints

Practitioners compare training checkpoints by weight deltas (coordinate-dependent) or by behavioral evaluations (coarse). Compute instead a Fisher–Rao-style distance between the output distributions of successive training checkpoints, restricted to a probe corpus. Does distance-per-step spike at known events - abrupt capability transitions during pretraining, or fine-tuning boundaries? An invariant odometer for training would let different runs and different parametrizations be compared on one axis.

Programs for Chapter 2

`kl_bowl_and_fisher.py` For a Bernoulli(p) family: numerically compute $D_{\mathrm{KL}}(p_\theta \| p_{\theta+\delta})$ for a grid of nudges δ at several base settings $\theta \in \{0.5, 0.8, 0.95\}$; overlay the quadratic $\frac{1}{2}F(\theta)\delta^2$ with the analytic $F = 1/p(1-p)$. Watch the bowl narrow as θ approaches the edge. Repeat for a Gaussian mean at several σ to see $F = 1/\sigma^2$. Two families, one law: curvature = Fisher. *Type: script + matplotlib. Deps: numpy.*

`fisher_rao_distance_bernoulli.py` Plot Euclidean distance $|p-q|$ and Fisher–Rao distance $2\arccos(\sqrt{pq} + \sqrt{(1-p)(1-q)})$ between coins, as heatmaps over (p, q) and as slices from $p = 0.5$. Mark the pairs $(0.50, 0.51)$ and $(0.98, 0.99)$: equal on the dial, far apart in distinguishability. Then verify the honest-ruler property empirically: for both pairs, plot how many coin flips a likelihood-ratio test needs to tell the machines apart at 95% confidence - the flip counts track the Fisher–Rao ruler, not the Euclidean one. *Type: script + matplotlib. Deps: numpy.*

3. Neural Networks as Probability Distribution Machines

Chapters 1 and 2 supplied a vocabulary: distributions to be shaped (Shannon's axis) and a geometry on the machines that shape them (Fisher's axis). This chapter introduces the machines themselves - and insists on a deliberately plain description of them. A generative neural network is a **probability distribution machine**: a parametrized function whose output is a probability distribution, trained to move distribution towards a target - typically in data or in instructions.

Structure of this chapter. Sections 3.1–3.3 cover the three architectural strategies for distribution-shaping. Section 3.1 covers **sequential conditioning**: the transformer as the current highest rung of Chapter 1's ladder, replacing the counted n-gram state with a learned, content-addressable state over the full context. Section 3.2 covers **global reshaping**: flow and diffusion models, which do not condition symbol-by-symbol at all but learn a transformation warping a simple base distribution directly into the data distribution. Section 3.3 covers **division of labor**: multi-head attention and mixture-of-experts routing, which split the shaping job across specialized sub-machines. Section 3.4 then draws the conclusion the first three sections force: the lenses of Chapters 1 and 2, powerful as they are, describe only the machine's outside - and understanding the inside requires new instruments. That closing section introduces the ideas of representations, circuits, and model biology. Throughout, the question to keep asking is not "what does the architecture look like" but "*what does it do to the distribution.*"

3.1 Transformers as Conditional Distribution Shapers

3.1.1 The ladder, resumed

Chapter 1 ended at the state-explosion wall: conditioning on long contexts by counting fails because every long context is new. The resolution is to stop counting and start *learning*: map each context to a vector in $\mathbb{R}^d$, and make similar contexts map to similar vectors, so that evidence from one context generalizes to contexts never seen. The ladder of conditioning strategies, extended to its current top rung:

Machine	**What conditions the next-symbol distribution**	**Memory handling**
Uniform	nothing	none
n-gram (Markov)	last k symbols, by counting	hard cutoff at k
Recurrent network	one compressed hidden vector, updated each step	unbounded in principle, lossy in practice
Transformer	learned attention over *all* previous tokens	full context, content-addressable

The transformer is best read as a generalized Markov machine. It still emits one conditional distribution per position: a softmax over the vocabulary, trained by the cross-entropy objective.

What changed from the previous rungs in the ladder is the state. An n-gram's state is the last k symbols, fixed in advance; a recurrent network's state is one vector that must compress everything; a transformer's state is assembled *on demand*: at each position, attention computes a weighted combination of all previous positions' representations, with weights determined by learned relevance (query–key match) rather than by recency. The state is content-addressable ("fetch whatever in the context matters for predicting the next token"), which is why transformers capture long-range dependencies that a fixed-window machine structurally cannot.

The model learns separately per attention head the weight matrix wQ : features describing what I need and the weight matrix wK : features describing what I offer.

The bilinear form formed by Q and K serves a few purposes. Consider each has dimensions say 4096×128 for a model. The product QKt has dimensions 4096x4096 but has a max rank of 128. Saving the two matrices separately instead of as their product requires 16x fewer parameters than their product (2x4096x128 vs 4096x4096). During inference, separating them enables Kx caching, which allows attention to be calculated using the KV cache.

3.1.2 The transformer decoder, component by component

The architecture used for language modeling is the multi-layer **transformer decoder** (Liu et al., 2018), a variant of the transformer (Vaswani et al., 2017), as specified for generative pretraining by Radford et al. (2018, §3.1): a multi-headed self-attention operation over the input context tokens followed by position-wise feedforward layers, producing an output distribution over target tokens. The complete computation is three equations. Given a context of k tokens $U = (u_{-k}, \ldots, u_{-1})$, written as a $k \times |V|$ matrix of one-hot rows:

$$\begin{aligned} h_0 &= UW_e + W_p \\ h_l &= \text{transformer\_block}(h_{l-1}) \quad \forall\, l \in [1, n] \\ P(u) &= \text{softmax}(h_n W_e^\top) \end{aligned}$$

where n is the number of layers, $W_e \in \mathbb{R}^{|V| \times d}$ is the token embedding matrix, and $W_p \in \mathbb{R}^{k \times d}$ is the position embedding matrix. The components, in order of application:

- **Token embedding** W_e. Row lookup mapping each vocabulary item to a vector in $\mathbb{R}^d$. Vocabulary is subword units (byte-pair encoding), not words or characters.
- **Position embedding** W_p. A learned vector per position, added to the token embedding. Necessary because attention is permutation-invariant: without W_p, the model could not distinguish token order. (Vaswani et al. used fixed sinusoids; the decoder of Radford et al. learns W_p.)
- **Masked multi-headed self-attention.** Each position computes query, key, and value projections of the layer input; attention weights are the softmax of scaled query–key dot products; the output at each position is the weight-averaged values. The *mask* sets weights on future positions to zero, so position i attends only to positions $\leq i$; this is what makes the model a valid autoregressive factorization. *Multi-headed*: the d dimensions are split across n_h heads that attend independently and are concatenated and linearly projected back to $\mathbb{R}^d$; head structure is treated in Section 3.3.
- **Position-wise feedforward network.** Two linear maps with a nonlinearity between (GELU in Radford et al.), inner width $4d$, applied identically and independently at every

position. Attention is the only component that moves information between positions; the feedforward network transforms each position in place.

- **Residual connections and layer normalization.** Each of the two sublayers is wrapped as $x \mapsto \text{LayerNorm}(x + \text{sublayer}(x))$: the sublayer's output is added to its input, then normalized. The additive path is what makes h_l an accumulating sum of sublayer writes rather than a chain of replacements.
- **Unembedding $W_e^\top$.** The final state h_n is projected back to vocabulary logits by the transpose of the input embedding (weights tied), and a softmax yields the next-token distribution. This is the q whose cross-entropy against data is the training objective of Section 1.3; equation (1) of Radford et al. is that objective, summed over positions.

Reference hyperparameters, for scale (Radford et al., 2018): $n = 12$ layers, $d = 768$, $n_h = 12$ heads, feedforward inner width 3072, context $k = 512$, byte-pair vocabulary of 40,000.

The **residual stream** is a running d-dimensional vector at each position - the h_l sequence - which every sublayer reads from and adds into; it is the shared channel through which intermediate results travel. The **attention head** is the smallest separable unit of the attention operation. Whatever the trained model computes, it computes by heads and feedforward layers writing into and reading from the residual stream; understanding a trained transformer means understanding what travels through that stream - a task taken up in Section 3.4.

3.2 Flow and Diffusion Models as Global Reshaping

3.2.1 Shaping without conditioning

Sequential conditioning is not the only way to place probability mass on structured outputs. The second strategy abandons the symbol-by-symbol factorization entirely: start from a distribution that is trivial to sample (a standard Gaussian) and learn a transformation T that warps it into the data distribution. Sampling is then one draw of noise pushed through T. Where the transformer shapes the distribution *one conditional at a time*, a flow model shapes it *all at once*, as a global change of variables:

$$p_{\text{model}}(x) \;=\; p_{\text{base}}\big(T^{-1}(x)\big)\left|\det \frac{\partial T^{-1}}{\partial x}\right|,$$

the Jacobian factor accounting for how the warp stretches and compresses probability mass - geometry doing the bookkeeping that conditional probabilities did in Chapter 1.

Two families realize the idea at scale. **Diffusion models** (Ho, Jain, and Abbeel, 2020) learn the warp implicitly: corrupt data by gradually adding Gaussian noise until nothing remains, train a network to undo one small step of corruption, and generate by iterating the denoiser from pure noise: a walk from the base distribution to the data distribution in hundreds of small steps. **Rectified flows** (Liu, Gong, and Liu, 2022) learn it explicitly as a velocity field: pair noise samples with data samples, draw straight lines between them, and train a network to follow those lines, so that generation is numerical integration of an ordinary differential equation - ideally in very few steps, since straight paths are cheap to integrate.

3.2.2 The contrast that matters

	Sequential (transformer)	**Global (flow/diffusion)**
Shaping mechanism	one learned conditional per symbol	one learned warp of the whole density
Where probability lives	explicit: softmax each step	implicit: base density × Jacobian
Sampling	token by token, left to right	noise in, sample out (possibly many refinement steps)
Natural domain	discrete sequences (text, code)	continuous spaces (images, audio, latents)
Chapter-1 ancestor	the n-gram ladder	none - a genuinely different lineage
Fisher-geometry connection	training objective is cross-entropy on conditionals	the warp itself is mass transport across the space

The point of including flows in this course: they demonstrate that "shaping a distribution" is the invariant task, with conditioning merely one strategy for it. A transformer sculpts the distribution through a factorization; a flow sculpts it through a map. Both end at the same place - a machine whose dials parametrize a distribution, whose quality Shannon's cross-entropy scores, and whose dial-space carries the Fisher geometry of Chapter 2. Training either machine is motion across that geometry, which is where Chapter 4 picks up.

3.3 Mixture-of-Experts and Multi-Head Structure

3.3.1 Dividing the shaping job

The third strategy is orthogonal to the first two: instead of changing *how* the distribution is shaped, divide *who* shapes it. Modern architectures split the work along two axes.

Multi-head attention (Vaswani et al., 2017) runs many small attention operations in parallel within each layer, each with its own learned query, key, and value projections - each head free to attend by different criteria (recency, syntax, coreference, matching brackets) and to read and write its own low-dimensional slice of the residual stream. The full layer's contribution is the sum of its heads: the conditional distribution is shaped by a *committee*, each member specializing in one kind of relevance.

Mixture-of-experts routing (Shazeer et al., 2017) applies the same idea to the feed-forward layers, at coarser grain and with a twist: of N expert sub-networks per layer, a learned gate activates only the top-k (often 2) per token. Parameters scale with N; compute scales with k. The gate makes computation *conditional*: different tokens literally flow through different sub-machines, so the distribution-shaping function is now a routed composition rather than a fixed pipeline.

3.3.2 Why the division of labor matters for understanding

The committee structure is not just an efficiency trick. It determines what units of analysis exist when you try to understand the trained machine:

- *Heads are separable.* One can silence a single head, or read its attention pattern, and ask what that one committee member does. Sometimes there is a clean answer: heads have

been found that track a token's predecessor, match closing brackets to their openers, or copy repeated patterns forward.

- *The residual stream is shared.* Many heads and layers write into the same d-dimensional workspace. What the model "knows" at any moment is therefore spread across the sum of many contributions, and no single neuron need correspond to any single concept.
- *Expert routing is discrete.* In a mixture-of-experts model, different tokens flow through different sub-networks. Any account of what the model does is conditional on the routing path taken.

The design lesson runs both directions. Architecture determines what kinds of explanation are even possible. And empirical findings about which specializations actually emerge reveal how the architecture's degrees of freedom are actually used.

Neuroscience parallel. Division of labor among specialized sub-machines is an organizational fact known about the cortex. Distinct brain regions handle faces, places, motion, and language. Lesion evidence established this in the 19th century. Functional imaging later mapped it noninvasively, using contrast designs: show faces versus objects, and the region that cares about the difference reveals itself (Kanwisher et al., 1997, for the fusiform face area). The corresponding objects in the artificial network are attention heads and routed experts: sub-networks recruited conditionally by input type. A shared caution transfers with the parallel. In both substrates, specialization is a matter of degree, and clean one-region-one-function stories usually dissolve on close inspection.

3.4 The Case for Interpretability

3.4.1 Two powerful lenses, one shared blind spot

Part I has equipped us with two lenses, and it is worth being precise about what each one sees. The Markov/Shannon lens describes the machine's *output*: a conditional distribution over the next symbol, scored by cross-entropy, improved by richer conditioning. The Fisher/Rao lens describes the machine's *dials*: how sensitively the output distribution responds to parameter changes, and what a principled distance in dial-space looks like.

Both lenses share a blind spot. Each treats the network as a single, undivided probability-emitting function. Shannon's quantities are computed from the output distribution alone; they say nothing about how attention heads, feed-forward layers, or experts produced that distribution. Fisher's geometry lives on the parameters as one undifferentiated vector; after training it does not decompose into statements like "this head, in this layer, handles subject–verb agreement."

Neither lens tells you where the fact is stored, which components retrieved it, what else is entangled with it, or how to change it without retraining the whole machine as their resolution is too coarse. New instruments are required for answering these questions.

3.4.2 Opening the machine: representations and circuits

The new instruments start from the two anatomical facts established in Sections 3.1 and 3.3, and turn each into an object of study.

Representations. At every token position, at every layer, there is an activation vector in the residual stream. These vectors are the model's working notes. A central empirical finding is that these notes have usable structure - in particular, many human-recognizable concepts (a language, a sentiment, a topic, the truth of a statement) correspond approximately to *directions* in activation space. This makes reading the notes possible by projecting the activation onto a direction and obtaining a measurement of a concept.

Circuits. The committee members of Section 3.3 (heads, layers, experts) connect through the residual stream into small functional assemblies: one head marks where a pattern occurred earlier, another copies what followed it, and together they implement "repeat what came after the last occurrence." Such an assembly is called a *circuit*: a subgraph of components implementing an identifiable algorithm. Circuits are the natural candidate for the level of description both lenses lack - coarser than a parameter, finer than the whole machine, and stated in terms of what is computed rather than merely what is output.

Interventions. Unlike any physical system of comparable interest, an artificial network is fully observable and fully writable: every activation can be recorded, every component silenced, every direction added or removed, mid-computation, at will. Hypotheses about representations and circuits can therefore be tested *causally* rather than argued from correlation: silence the candidate component and watch the behavior; edit the candidate direction and watch the concept.

3.4.3 Model biology

A trained network is not designed; it is *grown*. Billions of parameters are set by an optimization process. The right stance toward such an object is the biologist's stance toward an organism: observe behaviors, form competing hypotheses, intervene, and let the interventions decide. Neuroscience spent a century building this toolkit for brains (receptive-field mapping, lesion studies, population-level analysis), and some of it transfers along lessons about where each method misleads.

The questions are: what do the activation vectors represent, and how are they arranged? Which assemblies of components implement which behaviors? How can behavior be changed by editing the inside rather than retraining the whole? And when an explanation is offered, how do we validate it is true of the machine rather than merely plausible. These four questions (representation, circuits, control, and evidence) define the second half of this course.

Takeaways

- A generative network is a distribution machine; architectures differ in shaping strategy. The transformer is the learned successor of the n-gram ladder: a content-addressable Markov state over the full context, emitting one softmax conditional per position (Vaswani et al., 2017).
- Flow and diffusion models shape globally instead: a learned warp from base Gaussian to data distribution (Ho et al., 2020; Liu et al., 2022) - proof that conditioning is one strategy for the invariant task, not the task itself.

- Multi-head attention and mixture-of-experts routing divide the shaping job across specialized sub-machines (Shazeer et al., 2017), creating the units (heads, experts, the shared residual stream) that any account of the machine's inner workings must be written in.
- The Markov/Shannon and Fisher/Rao lenses see only the machine's outside: output distribution and parameter sensitivity. Understanding the inside requires new objects (representations, circuits, causal interventions) studied with a biologist's stance toward a grown, undocumented system.

Research Directions

From §3.1: Where does the learned state beat the counted one?

The transformer's advantage over n-grams must be localized in *which tokens* it predicts better. Compare a small transformer against order-2 through order-5 word models token-by-token on shared text; cluster the tokens where the transformer's advantage is largest. Do the clusters correspond to identifiable long-range computations - agreement across clauses, coreference, bracket matching? This builds a null model that any explanation of the machine needs: a mechanism claim is only interesting for behavior the counted state cannot already explain.

From §3.2: One geometry for two shaping strategies

Transformers and flows shape distributions by different means; do they arrive at similar internal structure? Train both a small autoregressive model and a small flow or diffusion model on the same low-dimensional distribution, then compare their internal representations, using dimensionality profiles of the activations and representational similarity analysis. Convergent structure across shaping strategies would suggest the data, not the architecture, dictates what forms inside - a foundational question for how far any analysis of one model family transfers to another.

From §3.3: Does routing buy cleaner components?

The cleanest architectural question raised by this chapter: for matched capability, do mixture-of-experts models pack fewer unrelated features into each neuron than dense models - i.e., does the router absorb specialization that a dense model must squeeze into shared dimensions? Train small dense and mixture-of-experts models on the same corpus and compare, per component, how many distinct input types drive each unit. Either answer matters: cleaner experts would make routing an implicit aid to understanding; equally mixed experts would show the packing pressure comes from something other than raw dimension scarcity.

Programs for Chapter 3

`ngram_vs_tiny_transformer.py` Train a 2-layer, 2-head character transformer (∼0.5M parameters, torch, one laptop-hour on CPU) on the Chapter-1 corpus, alongside order-2 through order-5 character Markov models. Report held-out per-character cross-entropy for all machines on one bar chart - the Chapter-1 entropy ladder extended by one learned rung. Then print the ten test positions where transformer and best n-gram disagree most, and look at them: they are almost always long-range dependencies. *Type: training script + matplotlib. Deps: torch, numpy.*

`flow_warp_2d.py` Implement a minimal 2-D normalizing flow (four affine coupling layers, ∼100 lines of torch) and train it to map a standard Gaussian to a two-moons distribution. Animate the warp: plot the image of a regular grid and of 1,000 base samples after each coupling layer, watching the Gaussian bend into two moons. Overlay the exact log-density from the change-of-variables formula. Global reshaping, watched frame by frame. *Type: training script + matplotlib animation. Deps: torch, numpy.*

`expert_specialization_toy.py` Build a mixture-of-experts regressor: four small multilayer perceptrons and a learned soft gate, trained on data drawn from four visibly different regimes (four quadrants of the plane with different functions). Plot (a) the gate's routing probabilities over the plane and (b) each expert's error restricted to each regime, across training. Watch specialization emerge from a uniform start - then break it: repeat with top-1 hard routing and observe expert collapse (one expert takes everything), the failure mode that load-balancing losses exist to prevent. *Type: training script + matplotlib. Deps: torch, numpy.*

4. Training Dynamics

Chapter 3 built machines whose dials parametrize probability distributions; Chapter 2 built the geometry of those dials. This chapter puts them together: *training is motion across the Fisher geometry.* That single sentence, unpacked carefully, explains a method that uses the geometry deliberately (natural gradient descent), the engineering that makes it affordable (Kronecker-factored approximation), and, most strikingly, why plain stochastic gradient descent, which never computes any of it, behaves as if it knew the geometry all along.

Structure of this chapter. Section 4.1 derives **natural gradient descent**: why "steepest descent" is ill-defined until you choose a metric, and what update rule falls out when the metric is Fisher's. Section 4.2 covers **tractability**: the full Fisher matrix for a billion-parameter model is unusable, and Kronecker factorization is the approximation that made curvature-aware training practical, with a look at how everyday optimizers like Adam smuggle in a diagonal version of the same idea. Section 4.3 turns to the empirical mystery: **plain stochastic gradient descent**, with no curvature computation at all, preferentially finds flat minima, and "flat," once the reparametrization objection is dealt with, must be measured in the Fisher-geometric terms of Chapter 2. Section 4.4 then confronts a question the first three sections raise but do not answer: if training dynamics build the structure inside the machine, can training be *chosen* so that the structure comes out legible and controllable? The research there sorts into three regimes: incidental influence, training for legibility, and training for steerability. The third is moving fastest. Claims in Sections 4.1–4.2 are theorems and engineering; Sections 4.3–4.4 are partly established, partly active research, and are flagged accordingly.

4.1 Natural Gradient Descent

4.1.1 Steepest descent is metric-dependent

Gradient descent updates parameters by $\Delta\theta = -\eta\nabla_\theta L$: step opposite the gradient. The usual justification ("the gradient is the direction of steepest ascent") hides an assumption. *Steepest per unit of what?* The gradient is steepest per unit of *Euclidean* parameter distance. But Chapter 2 established that Euclidean distance on dials is a dishonest ruler: a fixed-size step in parameter space produces wildly different amounts of change in the output distribution depending on where you stand and how the dials are labeled. Plain gradient descent therefore takes reckless steps where the distribution is hair-trigger sensitive and timid steps where it is loose - and its trajectory changes if you merely relabel the parameters.

Amari (1998) posed the sharper version of the question: which update direction gives the most loss reduction per unit of *distribution change* - per unit of Fisher–Rao distance, the invariant ruler of Section 2.2? Constrained steepest descent under $ds^2 = d\theta^\top F\, d\theta$ has a closed-form answer, the **natural gradient**:

$$\Delta\theta \;=\; -\eta\, F(\theta)^{-1}\nabla_\theta L.$$

Multiplying by the inverse Fisher matrix rescales every direction by its own sensitivity: loose dials (low curvature) get large steps, hair-trigger dials (high curvature) get small ones, and cor-

related dials are decorrelated. The result is invariant to reparametrization (percent-vowels and log-odds-of-vowels coordinates yield the same trajectory through distribution space), which plain gradient descent cannot claim (Martens, 2014, treats the modern theory thoroughly, including the equivalence to a generalized Gauss–Newton method and the sense in which natural gradient gives second-order convergence behavior).

4.1.2 A two-dial example

Fit a Gaussian to data by adjusting (μ, σ). The Fisher matrix is diagonal with entries $1/\sigma^2$ and $2/\sigma^2$: when σ is small, *both* dials are hair-triggers - a fixed nudge of the mean is large relative to σ, so the old and new distributions barely overlap. Plain gradient descent, blind to this, oscillates wildly as σ shrinks (the effective step size in distribution space grows as $1/\sigma^2$), forcing a tiny learning rate chosen for the worst region. Natural gradient multiplies by $F^{-1} = \mathrm{diag}(\sigma^2, \sigma^2/2)$ and walks smoothly at every scale. The program `natural_vs_plain_gradient.py` draws both trajectories, and the picture is worth the whole derivation: one path zigzags into the narrow-σ region and stalls; the other moves at constant speed in distribution space, following, approximately, a Fisher–Rao geodesic.

4.2 Making It Tractable: Kronecker-Factored Approximate Curvature

4.2.1 The impossible matrix

For a model with n parameters, F is $n \times n$. At $n = 10^9$, storing it needs 10^{18} numbers and inverting it $\sim 10^{27}$ operations - both absurd. Every practical use of Fisher geometry in deep learning is a story about approximating F^{-1} cheaply, and the approximations form a ladder of fidelity:

- **Diagonal.** Keep only F's diagonal - per-parameter sensitivities, ignoring correlations. This is where everyday optimizers live: Adam's per-parameter rescaling by root-mean-square gradients is, in expectation, a diagonal empirical-Fisher preconditioner. Nearly everyone who trains a network is thus running a crude natural gradient without calling it that.
- **Kronecker-factored (K-FAC).** Martens and Grosse (2015) observed that for one layer computing Wa (weights times incoming activations), the layer's Fisher block is, under reasonable assumptions, a Kronecker product $F_{\text{layer}} \approx A \otimes G$, where A is the (small) covariance of the layer's input activations and G the (small) covariance of the gradients flowing back into it. The Kronecker identity $(A \otimes G)^{-1} = A^{-1} \otimes G^{-1}$ converts one impossible inversion into two easy ones - for a 1000×1000 weight matrix, inverting two 1000×1000 factors instead of one $10^6 \times 10^6$ block. K-FAC converges in far fewer steps than plain stochastic gradient descent on the networks it was built for, at a per-step overhead of roughly two- to three-fold.
- **Full.** Only feasible for the toy models of this book's programs - which is what makes them instructive.

4.2.2 What curvature-aware training is for

A summary of practice: plain first-order methods, stabilized by normalization layers and tuned learning-rate schedules, won the large-scale pretraining race on wall-clock simplicity; curvature-aware methods persist where their per-step quality matters - reinforcement learning from human feedback and policy optimization (where Fisher preconditioning descends from trust-region methods), fine-tuning under tight step budgets, and settings with pathological conditioning. For this course, K-FAC's importance is conceptual as much as practical: it demonstrates that the Fisher geometry of Chapter 2 is not philosophy; it is an engineering resource that buys measurable convergence when paid for, which sharpens the puzzle of the next section: why does the method that *doesn't* pay for it still inherit its benefits?

4.3 Stochastic Gradient Descent's Implicit Geometric Bias

4.3.1 The flat-minima observation

A deep network's loss surface has many minima with near-identical training loss but very different test performance. The empirical regularity, documented sharply by Keskar et al. (2016) in the large-batch setting: minima found by small-batch stochastic gradient descent tend to be *flat* (the loss rises gently under parameter perturbation), and flat minima generalize better, while large-batch training tends toward sharp minima that generalize worse. The prevailing intuition: a minimum whose basin is wide is robust to the difference between training and test distributions, in the same way a machine sitting in a broad bowl of Chapter 2 keeps producing nearly the same output under parameter jitter.

4.3.2 The objection that makes it geometric

Dinh et al. (2017) broke the naive version of this story: flatness measured by the Hessian of the loss in raw parameter coordinates is *not reparametrization-invariant*. Rescale one layer's weights up and the next layer's down (leaving the network's function, hence its generalization, unchanged) and the same minimum can be made to look arbitrarily sharp or flat. Parameter-space flatness is an artifact of coordinates, and Chapter 2 taught precisely what to do with coordinate artifacts: replace the Euclidean ruler with the Fisher ruler. The repaired quantity is a *Riemannian* (Fisher-weighted) sharpness, schematically $\mathrm{tr}\big(F(\theta^*)^{-1}\nabla^2 L(\theta^*)\big)$, which measures the loss's curvature *per unit of distribution change* rather than per unit of dial-turn, is invariant to relabeling, and correlates with generalization where the raw Hessian version can be gamed. The flat-minima story survives, but only in Fisher coordinates: what matters is flatness *on the manifold of distributions*, not in parameter space. (Status: the invariance repair is settled mathematics; precisely which invariant sharpness best predicts generalization remains active research.)

4.3.3 Why plain stochastic gradient descent finds them

The remaining question is mechanism: stochastic gradient descent computes no Fisher matrix - why would it prefer Fisher-flat minima? The gradient noise is the leading account. Mini-batch gradients are noisy, and the noise is anisotropic: its covariance is approximately the empirical Fisher, largest along the directions where the output distribution is most sensitive. Training therefore behaves like a ball rolling on the loss surface while being shaken. The shaking is

strongest along the directions where the bowl walls are steepest. Narrow bowls eject the ball; wide bowls retain it. Settling into wide-in-Fisher-terms basins is then not a preference stochastic gradient descent computes but a steady state of its noise dynamics: an implicit regularizer arising from the same geometry the optimizer never explicitly touches. (Status: this account is supported by analyses of the noise covariance and by controlled experiments; a complete quantitative theory for realistic architectures is open.)

Neuroscience parallel. Training noise has a biological counterpart, and the counterpart is deliberate. Juvenile songbirds learn to sing by practicing thousands of times a day. They never repeat the song the same way twice; every rendition varies. That variability is not motor sloppiness. It is injected on purpose, by a dedicated forebrain nucleus called LMAN. Ölveczky, Andalman, and Fee (2005) proved this by direct intervention: silence LMAN, and the juvenile's song immediately becomes stereotyped. The bird stops exploring, and song learning stalls. The point is simple and strong - the variability *is* the learning mechanism, not noise on top of it. The corresponding object in network training is mini-batch gradient noise. In both systems, structured randomness drives the search across the space of solutions. And in both, removing it (large-batch training in one case, LMAN inactivation in the other) produces convergence that looks cleaner and learns worse.

4.4 Training-Time Levers on Interpretability

4.4.1 Structured is not legible

Sections 4.1–4.3 established that training dynamics fill the machine with structure: compressed, robust solutions, selected by the geometry of the noise. It is tempting to conclude that understanding the machine is therefore easy - the structure is right there. The conclusion is false, and it is worth being precise about why.

The optimizer compresses for its own objective, not for our comprehension. The clearest case: when the data contains more useful features than the network has dimensions, the loss-optimal solution *packs* many features into shared dimensions, tolerating small interference between them (Elhage et al., 2022, demonstrate this in miniature models, where the packing can be watched directly). Packing is compression. It is also the single biggest obstacle to reading the network unit by unit: a neuron that participates in many packed features responds to a bewildering mixture of inputs, and no one-neuron-one-meaning story can survive. The same implicit bias that creates the regularity creates the obfuscation.

The apt analogy is a compiled, optimized binary with the debugging symbols stripped. Such a binary is deeply structured (that is why reverse engineering it is possible at all), but the structure was arranged for execution, not for reading. Training dynamics guarantee there is something worth decompiling. They do not do the decompiling. That gap is what keeps interpretability an open problem, and it raises the question this section pursues: since training builds the structure, can training be chosen so the structure comes out readable? The research sorts into three regimes.

4.4.2 Regime 1: incidental - hyperparameters silently set the code

The first regime is the uncomfortable one: choices already being made for other reasons determine how legible the result is, and almost nobody measures the effect.

Weight decay can select the legible algorithm. Recall the delayed-generalization phenomenon called grokking: on small algorithmic tasks, a network first memorizes its training set, then, much later and abruptly, starts generalizing. Nanda et al. (2023) reverse-engineered what happens underneath on modular addition. During the long plateau, the network is quietly assembling a clean, general algorithm (based on Fourier components and trigonometric identities) while the memorized solution still drives behavior; weight decay then drives a "cleanup" phase that strips the memorization away, and the clean algorithm takes over. Remove weight decay and this transition largely fails to happen. A regularization constant decided whether the trained network contained a human-readable algorithm or an opaque lookup table.

Dropout breeds redundancy. Dropout trains every component to survive its neighbors' random failure. The plausible product is networks full of backup components - circuits that lie dormant until a primary pathway is damaged, then take over its function. Redundancy of this kind is directly hostile to causal analysis: silence a component, observe no change, and wrongly conclude it did nothing.

Width and data sparsity set the packing pressure. In the miniature models of Elhage et al. (2022), how much feature-packing occurs is controlled by the ratio of useful features to available dimensions and by how sparsely features occur - quantities set by architecture and data curation. Legibility per neuron is, in part, a capacity-planning outcome.

The summary of regime 1: interpretability outcomes ride on optimizer and architecture settings chosen with no thought to interpretability. The third research direction at the end of this chapter (nobody has checked whether models trained with different optimizers are equally analyzable) is this regime's open measurement problem.

4.4.3 Regime 2: deliberate - training for legibility

The second regime chooses training procedures whose products are readable by construction. The record shows real gains, a recurring cost, and one famous trap.

- **Changed activation functions.** Elhage et al. (2022, "Softmax Linear Units") replaced the standard MLP nonlinearity with one designed to encourage basis-aligned, one-meaning-per-neuron representations. It worked (the fraction of interpretable neurons rose), but with one important complication. Some features did not become interpretable; they *relocated*, hiding in a normalization step where the interpretability metric was not looking. This is the field's cautionary tale: optimize a proxy for legibility, and the computation may reorganize to satisfy the proxy while staying opaque.
- **Discrete bottlenecks.** Tamkin et al. (2023) train networks whose hidden states must pass through a codebook: each token's state becomes a small set of discrete codes. The codes are sparse, enumerable, and individually inspectable, and flipping them changes behavior predictably - legibility and a control surface, bought at a modest capability cost.
- **Interpretable-by-architecture models.** Backpack language models (Hewitt et al., 2023) constrain predictions to be non-negative weighted sums of non-contextual word-sense vectors. The decomposition is not discovered after training; it is the architecture. Editing a sense vector produces predictable global changes in behavior.

- **Engineering away the packing.** Jermyn et al. (2022) show, in miniature models, that training adjustments (initialization and bias choices) can push otherwise-identical networks into one-meaning-per-neuron solutions at little or no loss penalty - evidence that some of the packing of regime 1 is contingent on training details rather than forced by the task.

The consistent economics: legibility can be bought at training time, the price is some capability (the "interpretability tax"), and the SoLU episode shows the purchase must be audited - a metric improved is not the same as a machine understood. As of this writing, no frontier production model is trained primarily for legibility; post-hoc analysis dominates because the tax competes directly with capability scaling. Whether the tax is fundamental or an artifact of immature methods is open.

4.4.4 Regime 3: deliberate - training for steerability

The third regime is the youngest and currently the most active. Its question: rather than making every neuron readable, can training place *specific* concepts and capabilities where we want them - localized, linear, and therefore controllable?

The foundation is a fact about why control is possible at all. Behavior can be steered by adding vectors to a network's internal activations because many high-level concepts turn out to be encoded as linear directions, and there is theoretical work arguing that this linearity is itself a product of training: the implicit bias of gradient descent, combined with the log-odds structure of the next-token objective, provably yields linear concept encodings in simplified settings (Jiang et al., 2024). Steerability is not an architectural accident. It is an emergent property of the training dynamics of Sections 4.1–4.3 - which means it can, in principle, be engineered.

The recent research does so from three directions:

- **Placing capabilities.** Gradient routing (Cloud et al., 2024) masks gradients during training so that designated data (say, a capability you may later want to remove) updates only a designated subregion of the network. The capability is localized by construction; deleting or gating that subregion later removes it cleanly. Localization stops being something you hope to discover and becomes something you specify.
- **Steering generalization itself.** Fine-tuning on narrowly flawed data (insecure code) can produce broadly misaligned behavior, a phenomenon called *emergent misalignment* (Betley et al., 2025). Concept-ablation fine-tuning (Casademunt et al., 2025) applies an intervention *during* fine-tuning: project out activation directions corresponding to concepts the model should not use, with no change to the data or the loss. The model then generalizes differently; in their experiments this largely prevented emergent misalignment. The significance: the intervention controls not what the model does on the training data, but *which* of the many solutions consistent with that data the optimizer selects.
- **Preventative steering.** Persona vectors (Chen et al., 2025) identify directions controlling character traits (sycophancy, harmfulness, hallucination-proneness) and use them in two training-time ways: screening fine-tuning data by how far it would push the model along a trait direction, and *preventative steering*, where the trait direction is added during fine-tuning so the model absorbs the pressure without encoding the trait, then behaves normally when the vector is removed at deployment. Follow-on work in 2026 extends the program in both directions along the pipeline: tracing how persona directions form across *pretraining*

checkpoints (Moskvoretskii et al., 2026), asking when in training a trait becomes linearly represented and whether it could be shaped earlier; and inoculation adapters (Riché et al., 2026), which isolate unwanted generalization into a removable adapter module during fine-tuning, with fewer side effects than steering the whole network.

The through-line of regime 3 is a genuine inversion. In the standard picture, training produces an opaque artifact and analysis begins afterward, from outside. In these methods, analysis instruments (concept directions, probes, localization maps) operate *inside the training loop*, deciding what the optimizer is allowed to learn and where it is allowed to put it. Training-time control is upstream of every after-the-fact fix: a trait that was never encoded needs no steering vector, and a capability that was localized on purpose needs no search to find.

4.4.5 Closing the arc

Part I ends here, so state its conclusion carefully. The machine of Chapter 3, trained under the dynamics of this chapter, does not merely reach low loss. It is selected for robust, compressed solutions - and that selection guarantees there is structure inside worth finding. But structured is not legible: the compression is optimized against the loss, not for the reader, and by default it actively packs meaning into forms no unit-by-unit inspection can untangle. What this section adds is that the default is negotiable. Training choices, some incidental and some deliberate, set how readable and how controllable the product comes out, and the newest work moves the instruments of understanding into the training loop itself. Opening the trained machine, as Section 3.4 argued we must, is the project of the second half of this course; this chapter's final lesson is that how hard that project will be is partly decided before training ends.

Takeaways

- "Steepest descent" is undefined until a metric is chosen; under the Fisher metric the answer is the natural gradient $\Delta\theta = -\eta F^{-1}\nabla L$ (Amari, 1998) - reparametrization-invariant, sensitivity-scaled, and equal progress per unit of distribution change.
- The full Fisher matrix is unusable at scale; K-FAC's per-layer Kronecker factorization (Martens & Grosse, 2015) made curvature affordable, and Adam's diagonal rescaling is the same idea at the crudest rung - Fisher geometry hides inside everyday optimizers.
- Flat minima generalize better (Keskar et al., 2016), but only Fisher-weighted flatness survives the reparametrization objection (Dinh et al., 2017); stochastic gradient descent's anisotropic noise plausibly drives it into Fisher-flat basins without ever computing the geometry.
- Structured is not legible: the optimizer compresses for its loss, not for the reader, and its default packing of features into shared dimensions obstructs unit-by-unit understanding. But the default is negotiable: training choices set legibility incidentally (weight decay, dropout, width), can buy it deliberately at a capability cost (activation functions, discrete bottlenecks, interpretable architectures), and can place and suppress specific concepts on purpose (gradient routing, concept-ablation fine-tuning, preventative steering) - moving the instruments of understanding inside the training loop.

Research Directions

From §4.1: Where does natural gradient still pay?

Run matched-budget comparisons of Adam, K-FAC, and plain stochastic gradient descent on small transformer pretraining (10–100M parameters), reporting loss against both step count and wall-clock. The literature's verdict is mixed and mostly dated; a careful modern accounting on one controlled setup (including the fine-tuning and reinforcement-learning regimes where curvature methods are claimed to win) would be immediately useful, and doubles as a measurement of how much of the Fisher geometry's promised benefit normalization layers and schedule tuning already capture for free.

From §4.3: Measure the invariant sharpness of real checkpoints

Compute Fisher-weighted sharpness (via Hessian- and Fisher-vector products, tractable without forming matrices) for checkpoints along real training runs, across batch sizes, learning rates, and abrupt delayed-generalization transitions on small algorithmic tasks (the phenomenon known as grokking). Does invariant sharpness fall when generalization improves, and does it *lead* or *lag* behavioral change? A leading relationship would make it a genuine early-warning channel for capability formation - an internal observable that moves before behavior does.

From §4.2–4.3: Does the optimizer's geometry shape the learned representation?

Train identical small models with plain stochastic gradient descent, Adam, and K-FAC to matched loss, then compare their internal structure: dimensionality profiles of the activations, how many distinct input types drive each unit, and the quality of features recovered by sparse dictionary methods. Do curvature-aware optimizers produce measurably different internal structure (cleaner features, different packing), or does the data distribution dominate? Either answer matters: analyses of one model are routinely assumed to transfer to models trained with different optimizers, and nobody has checked whether that transfer is licensed.

From §4.4: Audit a legibility intervention for hiding

The Softmax Linear Units episode showed that training for a legibility metric can relocate opaque computation instead of removing it. No standard audit exists for this failure mode. Take one train-for-legibility method (a codebook bottleneck, or an activation-function change) on a small model, and measure legibility *everywhere* - not just at the intervened component: per-neuron interpretability in the MLPs, but also in normalization parameters, attention outputs, and the residual stream. The deliverable is a conservation analysis: did total opacity fall, or did it move? A reusable audit protocol here would make every regime-2 result more trustworthy, and a demonstrated hiding effect in a new setting would be a finding in its own right.

Programs for Chapter 4

`natural_vs_plain_gradient.py` Fit a Gaussian's $(\mu, \log\sigma)$ to sample data by gradient descent on negative log-likelihood, from a deliberately bad start. Run plain gradient descent and natural gradient (the 2×2 Fisher matrix is analytic - invert it exactly). Plot both trajectories in (μ, σ) space over the loss contours, plus loss-vs-step. The zigzag-and-stall versus smooth-geodesic contrast is the whole of Section 4.1 in one figure. Then relabel the dial ($\sigma \to \sigma^2$) and rerun both: the natural-gradient trajectory through distribution space is unchanged; plain gradient's changes. Invariance, demonstrated rather than asserted. *Type: script + matplotlib. Deps: numpy only.*

`preconditioning_comparison.py` Logistic regression on synthetic data with feature scales spanning 10^3 (condition number $\sim 10^6$). Train with: plain stochastic gradient descent, Adam, diagonal-Fisher preconditioning, and exact natural gradient (feasible at this size). Plot loss-vs-step for all four. Adam and diagonal-Fisher nearly coincide (Adam *is* a diagonal Fisher method in disguise), while exact natural gradient shows what the full geometry buys and plain descent shows life without any. *Type: script + matplotlib. Deps: numpy, torch optional.*

`flat_minima_and_noise.py` Construct a 2-D loss surface with two global minima (one wide basin, one narrow) and run gradient descent with tunable isotropic noise from many random starts. Plot the fraction of runs ending in the wide basin as a function of noise scale: near zero noise, arrivals split by basin of attraction; with noise, the narrow basin empties. Then make it geometric: define train loss and a slightly shifted test loss, and show end-of-training test loss tracks basin width. The noise-selects-flatness mechanism of §4.3, small enough to see all at once. *Type: script + matplotlib. Deps: numpy only.*

References

Papers marked with a filename in brackets are archived as PDFs in `new_book/references/`.

Chapter 1 - Statistical Foundations

1. Markov, A.A. (1913). *An example of statistical investigation of the text "Eugene Onegin" concerning the connection of samples in chains.* Bulletin of the Imperial Academy of Sciences of St. Petersburg. (English translation: Science in Context 19(4), 2006.)

2. Gamow, G. (1947). *One Two Three… Infinity: Facts and Speculations of Science.* Viking Press.

3. Shannon, C.E. (1948). *A Mathematical Theory of Communication.* Bell System Technical Journal 27. [`shannon1948_mathematical_theory_communication.pdf`]

4. Shannon, C.E. (1951). *Prediction and Entropy of Printed English.* Bell System Technical Journal 30(1).

5. Barlow, H.B. (1961). *Possible principles underlying the transformation of sensory messages.* In *Sensory Communication*, MIT Press.

6. Laughlin, S. (1981). *A simple coding procedure enhances a neuron's information capacity.* Zeitschrift für Naturforschung C 36(9–10).

Chapter 2 - Geometric Foundations

7. Nielsen, F. (2018). *An Elementary Introduction to Information Geometry.* arXiv:1808.08271. [`nielsen2018_information_geometry_intro.pdf`]

8. Chentsov, N.N. (1972). *Statistical Decision Rules and Optimal Inference.* Nauka (English translation: AMS, 1982).

9. Seung, H.S., Sompolinsky, H. (1993). *Simple models for reading neuronal population codes.* Proceedings of the National Academy of Sciences 90(22).

Chapter 3 - Neural Networks as Distribution Machines

10. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention Is All You Need.* NeurIPS 2017. arXiv:1706.03762. [`vaswani2017_attention.pdf`]

11. Radford, A., Narasimhan, K., Salimans, T., Sutskever, I. (2018). *Improving Language Understanding by Generative Pre-Training.* OpenAI technical report. [`../language_understanding_paper.pdf`]

12. Liu, P.J., Saleh, M., Pot, E., et al. (2018). *Generating Wikipedia by Summarizing Long Sequences.* ICLR 2018. arXiv:1801.10198. (The decoder-only transformer variant.)

13. Ho, J., Jain, A., Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models.* NeurIPS 2020. arXiv:2006.11239. [`ho2020_denoising_diffusion.pdf`]

14. Liu, X., Gong, C., Liu, Q. (2022). *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow.* ICLR 2023. arXiv:2209.03003. [`liu2022_rectified_flow.pdf`]

15. Shazeer, N., Mirhoseini, A., Maziarz, K., et al. (2017). *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.* ICLR 2017. arXiv:1701.06538. [`shazeer2017_mixture_of_experts.pdf`]

16. Kanwisher, N., McDermott, J., Chun, M.M. (1997). *The fusiform face area: a module in human extrastriate cortex specialized for face perception.* Journal of Neuroscience 17(11).

Chapter 4 - Training Dynamics

17. Amari, S. (1998). *Natural Gradient Works Efficiently in Learning.* Neural Computation 10(2).

18. Martens, J. (2014). *New Insights and Perspectives on the Natural Gradient Method.* JMLR 21 (2020). arXiv:1412.1193. [`martens2014_natural_gradient_insights.pdf`]

19. Martens, J., Grosse, R. (2015). *Optimizing Neural Networks with Kronecker-factored Approximate Curvature.* ICML 2015. arXiv:1503.05671. [`martens2015_kfac.pdf`]

20. Keskar, N.S., Mudigere, D., Nocedal, J., Smelyanskiy, M., Tang, P.T.P. (2016). *On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima.* ICLR 2017. arXiv:1609.04836. [`keskar2016_large_batch_sharp_minima.pdf`]

21. Dinh, L., Pascanu, R., Bengio, S., Bengio, Y. (2017). *Sharp Minima Can Generalize For Deep Nets.* ICML 2017. arXiv:1703.04933. [`dinh2017_sharp_minima_can_generalize.pdf`]

22. Ölveczky, B.P., Andalman, A.S., Fee, M.S. (2005). *Vocal experimentation in the juvenile songbird requires a basal ganglia circuit.* PLoS Biology 3(5).

Chapter 4, §4.4 - Training-Time Levers on Interpretability

23. Elhage, N., Hume, T., Olsson, C., et al. (2022). *Toy Models of Superposition.* Transformer Circuits Thread / arXiv:2209.10652. [`elhage2022_toy_models_superposition.pdf`]

24. Nanda, N., Chan, L., Lieberum, T., Smith, J., Steinhardt, J. (2023). *Progress measures for grokking via mechanistic interpretability.* ICLR 2023. arXiv:2301.05217. [`nanda2023_grokking_progress_measures.pdf`]

25. Elhage, N., Hume, T., Olsson, C., et al. (2022). *Softmax Linear Units.* Transformer Circuits Thread, `transformer-circuits.pub/2022/solu`. (Web only.)

26. Tamkin, A., Taufeeque, M., Goodman, N.D. (2023). *Codebook Features: Sparse and Discrete Interpretability for Neural Networks.* ICML 2024. arXiv:2310.17230. [tamkin2023_codebook_features.pdf]

27. Hewitt, J., Thickstun, J., Manning, C.D., Liang, P. (2023). *Backpack Language Models.* ACL 2023. arXiv:2305.16765. [hewitt2023_backpack_language_models.pdf]

28. Jermyn, A.S., Schiefer, N., Hubinger, E. (2022). *Engineering Monosemanticity in Toy Models.* arXiv:2211.09169. [jermyn2022_engineering_monosemanticity.pdf]

29. Jiang, Y., Rajendran, G., Ravikumar, P., Aragam, B., Veitch, V. (2024). *On the Origins of Linear Representations in Large Language Models.* ICML 2024. arXiv:2403.03867. [jiang2024_origins_linear_representations.pdf]

30. Cloud, A., Goldman-Wetzler, J., Wybitul, E., Miller, J., Turner, A.M. (2024). *Gradient Routing: Masking Gradients to Localize Computation in Neural Networks.* arXiv:2410.04332. [cloud2024_gradient_routing.pdf]

31. Casademunt, H., Juang, C., Karvonen, A., Marks, S., Rajamanoharan, S., Nanda, N. (2025). *Steering Out-of-Distribution Generalization with Concept Ablation Fine-Tuning.* arXiv:2507.16795. [casademunt2025_concept_ablation_finetuning.pdf]

32. Chen, R., Arditi, A., Sleight, H., Evans, O., Lindsey, J. (2025). *Persona Vectors: Monitoring and Controlling Character Traits in Language Models.* arXiv:2507.21509. [chen2025_persona_vectors.pdf]

33. Moskvoretskii, V., Glandorf, D., Medina Moreira, J., Käser, T., West, R. (2026). *Tracing Persona Vectors Through LLM Pretraining.* arXiv:2605.13329. [moskvoretskii2026_tracing_persona_vectors.pdf]

34. Riché, M., Tan, D., Kohonen, V., Warncke, N., et al. (2026). *Inoculation Adapters: Improved Selective Generalization of Capabilities with Fewer Surprising Backdoors.* arXiv:2606.30252. [riche2026_inoculation_adapters.pdf]

35. Betley, J., Tan, D., Warncke, N., et al. (2025). *Emergent Misalignment: Narrow fine-tuning can produce broadly misaligned LLMs.* arXiv:2502.17424. [betley2025_emergent_misalignment.pdf]

Continued in Part II

36. The references for Chapters 5–8 (representation geometry, sparse autoencoders, circuits, steering, faithfulness, and the neuroscience lineage) are collected in Part II (`new_book/book/`), with archived PDFs indexed in `new_book/references/README.md`.